College of Engineering and Technology - Smart Village

Computer Engineering Department

ECE3502
Systems Programming

Lecture 4 – Compilers

3. Parsing Techniques

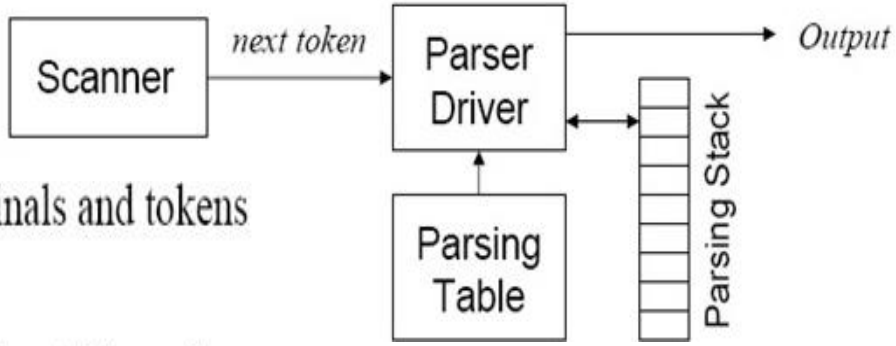
Parsing techniques are used to **check if the input string** is **legal** or **illegal** **using the CFG**. There exist two types of parsers include top-down and bottom-up:

1) **Top-down Parsers**
   - LL Parser
2) **Bottom-up Parsers**
   - LR parser

Top-down parser (LL parser)

First L means, the input string is scanned from left to right.
Second L means left most derivation used in parsing.

The top-down parser **consists of parser stack**, **parsing table** , and **driver function**.



- **Parser stack** that holds grammar symbols: non-terminals and tokens
- **Parsing table** that specifies the parser action
- **Driver function** that interacts with parser stack, parsing table and scanner

The LL parser actions are: predict , match, accept, and error

- **Match**: to match top of parser stack with next input token
- **Predict**: to predict a production and apply it in a derivation step
- **Accept**: to accept and terminate the parsing of a sequence of tokens
- **Error**: to report an error message when matching or prediction fails

Top-down Parser Algorithm

- **Step 1:** Start with the start symbol in stack
- **Step 2:** Scan the input string from left to right
- **Step 3:** Choose the closest production that matches to the input string
- **Step 4:** Compare the current terminal of the input string with the top of the stack
- **Step 5:** If they are matched then
  - move to the next terminal of the input
  - pop this terminal from the stack for each non-terminal child node, repeat step 2 until the string is generated or until no other productions are left.
- **Step 6:** if the input string is successfully scanned and the stack is empty then the string is legal. Otherwise, the string is illegal

Top-down Parser Algorithm

Example 1:

Can you Solve this problem?

Given the following Grammar

$$S \rightarrow (S) S \mid \delta$$

Check if the string : **(())** is legal or illegal using LL-parser

Solution

Parser Stack	Input	Parser Action
S	$(()) \$$	Predict $S \rightarrow (S) S$
$(S) S$	$(()) \$$	Match (
$S) S$	$()) \$$	Predict $S \rightarrow (S) S$
$(S) S) S$	$()) \$$	Match (
$S) S) S$	$)) \$$	Predict $S \rightarrow \delta$
$) S) S$	$)) \$$	Match)
$S) S$	$) \$$	Predict $S \rightarrow \delta$
$) S$	$) \$$	Match)
S	$\$$	Predict $S \rightarrow \delta$
Empty	$\$$	Accept

Since **all tokens of the input string is successfully scanned** and the **parser stack is empty**. **Therefore**, the string **(())** is **legal** (*part of the language generated from the above CFG*).

Top-down Parser Algorithm

Example 2:

Given the following Grammar

1: $E \rightarrow TQ$	6: $R \rightarrow *FR$
2: $Q \rightarrow +TQ$	7: $R \rightarrow /FR$
3: $Q \rightarrow -TQ$	8: $R \rightarrow \delta$
4: $Q \rightarrow \delta$	9: $F \rightarrow (E)$
5: $T \rightarrow FR$	10: $F \rightarrow$ **id**

Can you Solve this problem?

Check if the string : **id * (id + id)** is legal or illegal using LL- parser

Solution

Parser Stack	Input	Parser Action
E	id*(id+id)$	Predict $E \rightarrow TQ$
TQ	id*(id+id)$	Predict $T \rightarrow FR$
FRQ	id*(id+id)$	Predict $F \rightarrow$ id
id RQ	id*(id+id)$	Match id
RQ	*(id+id)$	Predict $R \rightarrow *FR$
$*FRQ$	*(id+id)$	Match *
FRQ	(id+id)$	Predict $F \rightarrow (E)$
$(E)RQ$	(id+id)$	Match (
$E)RQ$	id+id)$	Predict $E \rightarrow TQ$
$TQ)RQ$	id+id)$	Predict $T \rightarrow FR$
$FRQ)RQ$	id+id)$	Predict $F \rightarrow$ id
id $RQ)RQ$	id+id)$	Match id
$RQ)RQ$	+id)$	Predict $R \rightarrow \delta$
$Q)RQ$	+id)$	Predict $Q \rightarrow +TQ$
$+TQ)RQ$	+id)$	Match +
$TQ)RQ$	id)$	Predict $T \rightarrow FR$
$FRQ)RQ$	id)$	Predict $F \rightarrow$ id
id $RQ)RQ$	id)$	Match id
$RQ)RQ$	)$	Predict $R \rightarrow \delta$
$Q)RQ$	)$	Predict $Q \rightarrow \delta$
$)RQ$	)$	Match)
RQ	$	Predict $R \rightarrow \delta$
Q	$	Predict $Q \rightarrow \delta$
Empty	$	Accept

Since **all tokens of the input string is successfully scanned** and the **parser stack is empty**, therefore, the string **id * (id+id)** is **legal** (*part of the language generated from the above CFG*)

Top-down Parser Algorithm

Example 3:

Given the following Grammar

1.	<goal>	::=	<expr>
2.	<expr>	::=	<expr> + <term>
3.		|	<expr> – <term>
4.		|	<term>
5.	<term>	::=	<term> * <factor>
6.		|	<term> / <factor>
7.		|	<factor>
8.	<factor>	::=	num
9.		|	id

Check if the string : **x-2*y** is legal or illegal using LL-parser

Can you Solve this problem?

Since **all tokens of the input string is successfully scanned** and the **parser stack is empty**, so the string **x-2*y** is **legal** (*part of the language generated from the above CFG*)

Solution

Rule #	Stack	Input				
–	⟨goal⟩	↑x	–	2	*	y
1	⟨expr⟩	↑x	–	2	*	y
2	⟨expr⟩ + ⟨term⟩	↑x	–	2	*	y
4	⟨term⟩ + ⟨term⟩	↑x	–	2	*	y
7	⟨factor⟩ + ⟨term⟩	↑x	–	2	*	y
9	id + ⟨term⟩	↑x	–	2	*	y
–	+ ⟨term⟩	x	↑–	2	*	y
–	⟨expr⟩	↑x	–	2	*	y
3	⟨expr⟩ – ⟨term⟩	↑x	–	2	*	y
4	⟨term⟩ – ⟨term⟩	↑x	–	2	*	y
7	⟨factor⟩ – ⟨term⟩	↑x	–	2	*	y
9	id – ⟨term⟩	↑x	–	2	*	y
–	– ⟨term⟩	x	↑–	2	*	y
–	⟨term⟩	x	–	↑2	*	y
5	⟨term⟩ * ⟨factor⟩	x	–	↑2	*	y
7	⟨factor⟩ * ⟨factor⟩	x	–	↑2	*	y
8	num * ⟨factor⟩	x	–	↑2	*	y
–	* ⟨factor⟩	x	–	2	↑*	y
–	⟨factor⟩	x	–	2	*	↑y
9	id	x	–	2	*	↑y
–	Empty	x	–	2	*	y

Cannot match

Start over